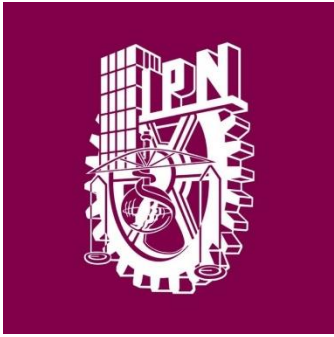




Instituto Politécnico
Nacional
Escuela Superior de
Computo



Profesor: Cortes Galicia Jorge

Materia: Sistemas Operativos

Practica 2

Integrantes:

Vázquez Blancas Cesar Said

Diaz Torres Jonathan Samuel

Aparicio Ponce Roberto Manuel

Corona Hernández Martin Rafael

Introducción Teórica

Los sistemas operativos son una parte fundamental para la correlación entre el usuario y el software, pues los sistemas operativos se encargan de manejar los recursos tanto del software como del hardware, por ellos es que administran las tareas y la administración del tiempo en los procesos, proporcionando una interfaz entre el usuario y el hardware; a pesar de que existen muchos sistemas operativos, de lado de la computadora hay dos muy famosos y conocidos los cuales son Windows y Linux.

Para entender fundamentalmente el concepto de los sistemas operativos es importante entender estas funciones clave:

- Gestión de procesos en función del tiempo.
- Gestión del uso de memoria RAM para asegurar el espacio suficiente.
- Controlar la comunicación entre los dispositivos hardware y software.
- Estructura para almacenamiento y recuperación de datos.
- Interfaz del usuario y su comunicación con el hardware.

Sistema operativo Windows:

Es un sistema operativo muy comercial y de código cerrado, lo que lo hace dominador en el mercado de máquinas de escritorio es que cuenta con una interfaz amigable para todos los usuarios.

Cuenta con:

- **Interfaz gráfica de usuario (GUI):** desde su versión 95 hasta la actualidad su interfaz ha sido mejorada con cada versión mas reciente, esto para facilitar la interacción con el usuario.
- **Sistema de archivos NTFS:** Windows utiliza el sistema de archivos NTFS lo que permite permisos avanzados, compresión, cifrado y soporte para grandes archivos.
- **Soporte extensivo de aplicaciones:** Windows permite instalar la mayoría de las aplicaciones comerciales, lo que hace que estén diseñadas para ejecutarse sin problema en este sistema operativo y poder darles un uso cotidiano.
- **Actualizaciones:** Windows ofrece actualizaciones de seguridad óptimas para cada máquina de escritorio, así como un soporte a largo plazo.

Sistema operativo Linux:

Sistema operativo basado en el kernel de Unix y es de código abierto, es decir que su código está abierto para su modificación y distribución, lo que lo ha hecho que avance en el terreno de las máquinas de escritorio, sistemas embebidos, servidores de dominio alterno.

Cuenta con:

- **Multitarea y multiusuario:** este sistema operativo está hecho desde sus inicios para soportar múltiples tareas y usuarios.
- **Seguridad:** protege el ingreso a archivos y recursos mediante permisos en grupo y filtros del sistema.
- **Servidores:** Linux es usado para servidores avanzados y sistemas críticos debido a que cuenta con una gran estabilidad, eficiencia, seguridad, robustez y capacidad para mantenerse en funcionamiento durante largos periodos de tiempos y exigencia de procesos.
- **Interfaz de línea de comandos (CLI):** Linux ofrece interfaces gráficas, su potencia y flexibilidad se encuentra en la CLI, lo que hace que permita una personalización y un control avanzados del sistema.

Diferentes tipos de sistemas operativos existen y cada uno tiene su uso particular para diferentes tipos de demandas en cada usuario, por ejemplo:

- **Monousuario:** MS-DOS, Windows 95
- **Multiusuario:** Unix, Linux, Windows Server
- **Tiempo Real (RTOS):** VxWorks, QNX, FreeRTOS
- **Distribuidos:** Plan 9, Apache Hadoop
- **Multiprocesador:** Linux SMP, Windows Server
- **Tiempo Compartido:** Unix, Multics
- **Embebidos:** FreeRTOS, Embedded Linux
- **Red:** Novell NetWare, Windows NT Server
- **Móvil:** Android, iOS
- **De Máquina Virtual:** VMware ESXi, Microsoft Hyper-V

1. Cree un archivo de texto (con cualquier contenido) y un archivo en Word (con cualquier contenido) en el sistema operativo Windows y guárdelo en una memoria usb.

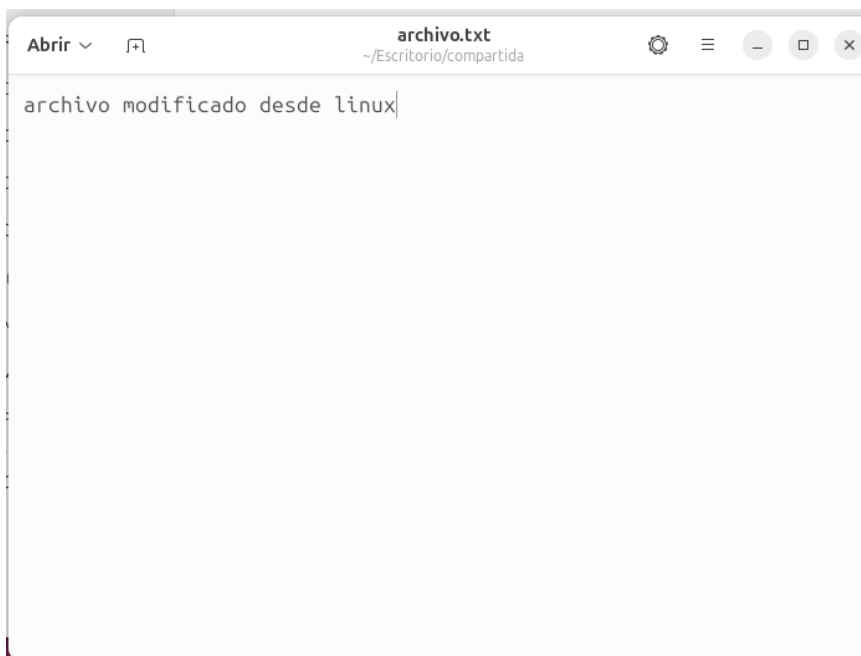
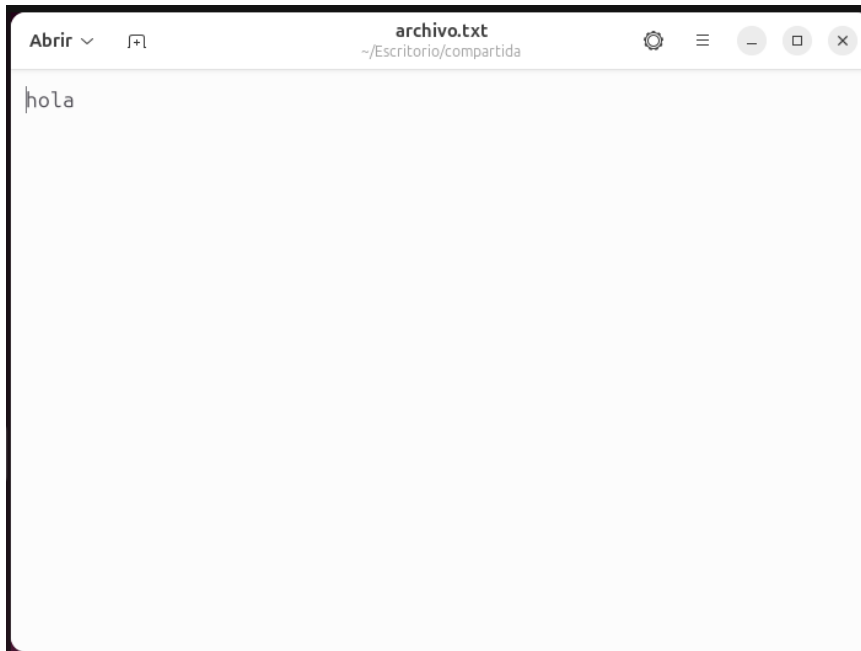


2. Inicie sesión en Linux

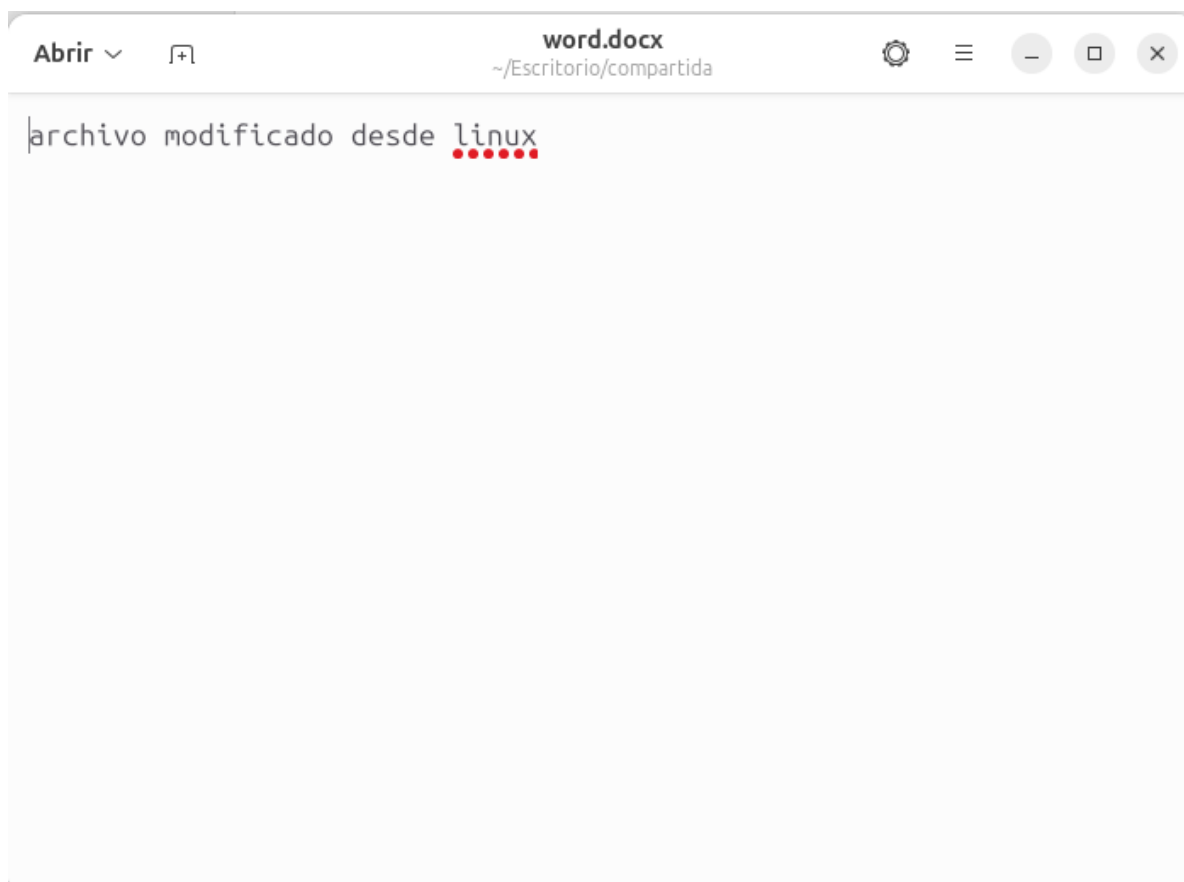
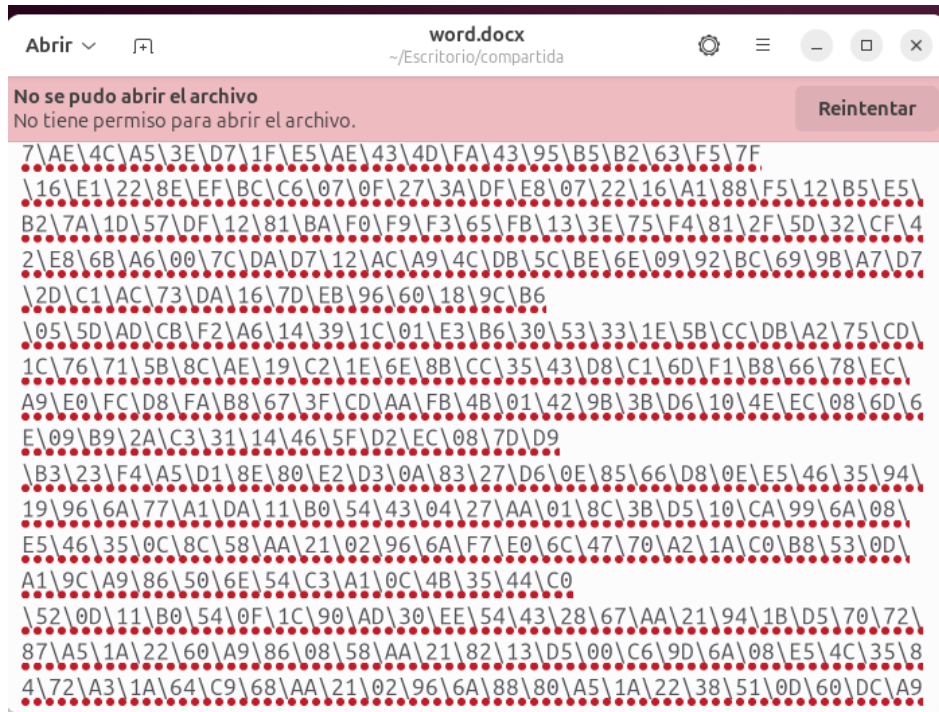
3. Verifique si está montada la unidad de memoria usb en su sistema, para ello introduzca una memoria usb y observe si es reconocida en el escritorio. Nota: En algunas distribuciones de Linux no está montada automáticamente la unidad de memoria usb, sino está montada la unidad usb, móntela mediante la terminal de la siguiente manera: Cree un directorio en /media/Nombre_Memoria, donde Nombre_Memoria es el nombre que quiera darle a su directorio. Introduzca una memoria usb en la unidad. Escriba: `mount -t vfat /dev/sdX /media/Nombre_Memoria`, donde en sdX la X representa el número que el sistema le asigna a su memoria usb. Escriba `cd /media/Nombre_Memoria`, si el cambio de directorio es realizado, su unidad de usb ha sido montada con éxito y puede continuar en el punto 4.

4. Edite tanto el contenido del archivo de texto como de Word, modificándolo mediante el uso de gedit. Guarde sus archivos.

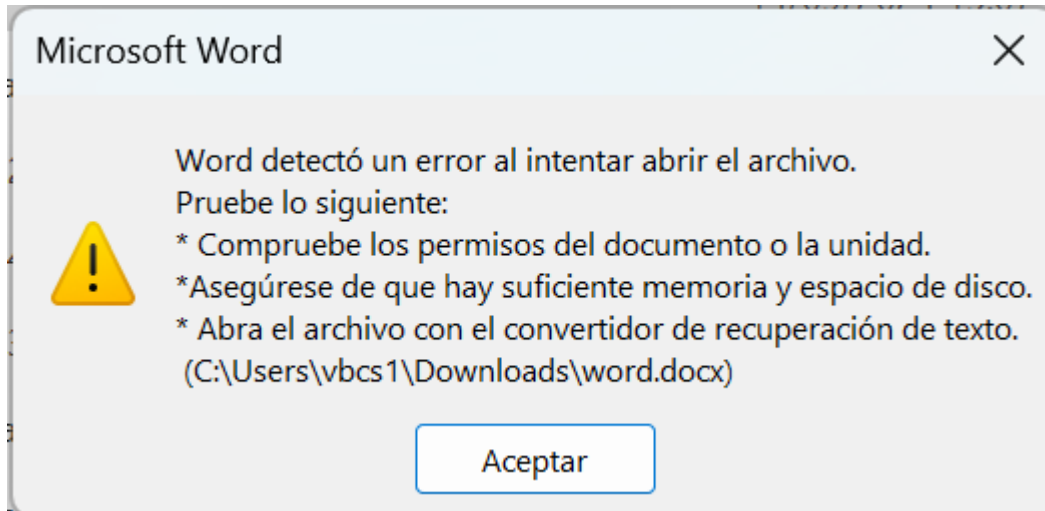
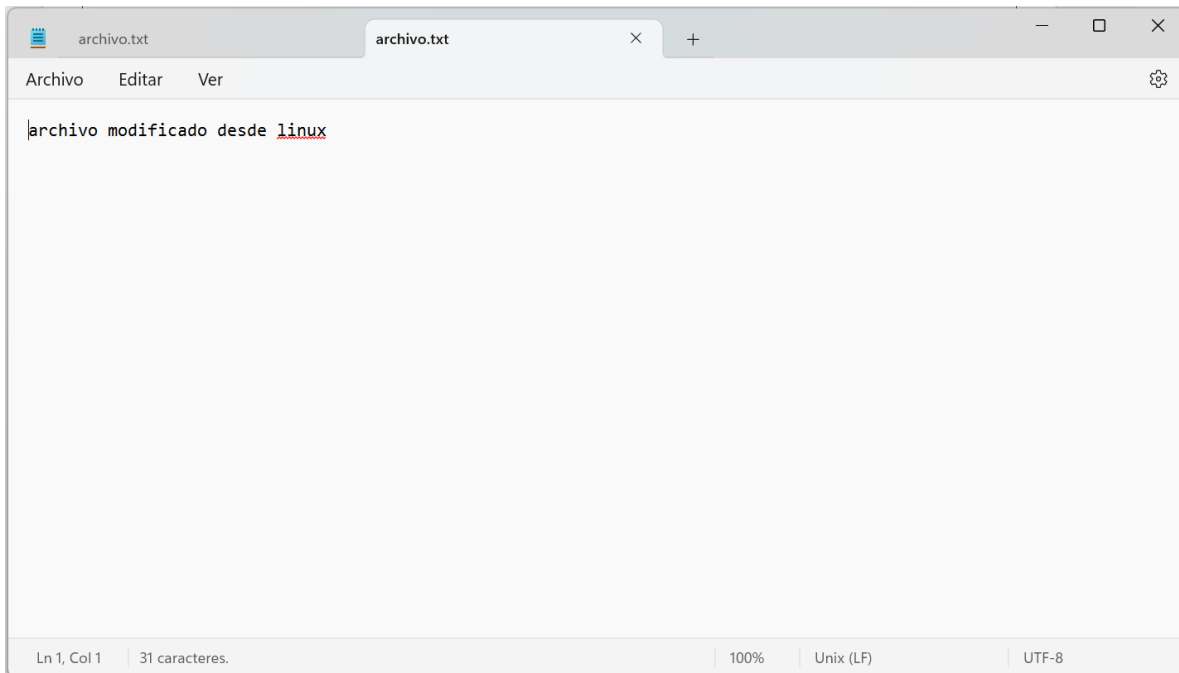
```
said@said-VirtualBox:~/Escritorio/compartida$ gedit archivo.txt
```



```
said@said-VirtualBox:~/Escritorio/compartida$ gedit word.docx
```



5. Inicie sesión en Windows y observe el contenido de sus archivos en su memoria usb. ¿Se observan las modificaciones realizadas en Linux?, explique el por qué si o no se observan.



En el archivo txt si se puede ya que son muy básicos, son texto sin ningún formato avanzado, por lo tanto se pueden abrir en cualquier sistema operativo

En el archivo Word parece que esta corrompido o dañado, esto pq al intentar editarlo con un editor de texto como vim o nano, al ser el Word un archivo binario, estos no pueden interpretar bien la estructura del archivo, alterando la codificación y estructura binaria del word

6. A través de su manual en línea de Linux (man) en la segunda o tercera sección (man 2 o man 3), investigue y reporte para qué sirven las siguientes llamadas al sistema de Linux: open, close, read, write, creat, lseek, access, stat, chmod, chown, fcntl, chdir, mkdir, opendir, y readdir.

1. open

- **Propósito:** Abre un archivo o dispositivo para su acceso (lectura, escritura o ambos).
- **Sintaxis:**

```
int open(const char *pathname, int flags);
```

```
int open(const char *pathname, int flags, mode_t mode);
```

- **Parámetros:**
 - pathname: Ruta del archivo que se desea abrir.
 - flags: Especifica cómo se debe abrir el archivo (lectura, escritura, etc.).
 - mode: Permisos de archivo (usado cuando se crea un archivo nuevo).

2.-close

- **Propósito:** Cierra un descriptor de archivo previamente abierto.
- **Sintaxis:**

```
int close(int fd);
```

- **Parámetros:**
 - fd: Descriptor de archivo que se desea cerrar.

3. read

- **Propósito:** Lee datos de un archivo referenciado por un descriptor de archivo.
- **Sintaxis:**

```
ssize_t read(int fd, void *buf, size_t count);
```

- **Parámetros:**
 - fd: Descriptor de archivo desde donde se leerá.

- buf: Buffer donde se almacenarán los datos leídos.
- count: Número máximo de bytes a leer.

4. write

- **Propósito:** Escribe datos en un archivo referenciado por un descriptor de archivo.
- **Sintaxis:**

```
ssize_t write(int fd, const void *buf, size_t count);
```

- **Parámetros:**
 - fd: Descriptor de archivo donde se escribirá.
 - buf: Datos a escribir.
 - count: Número de bytes a escribir.

5. creat

- **Propósito:** Crea un nuevo archivo o lo trunca si ya existe.
- **Sintaxis:**

```
int creat(const char *pathname, mode_t mode);
```

- **Parámetros:**
 - pathname: Ruta del archivo a crear.
 - mode: Permisos del nuevo archivo.

6. lseek

- **Propósito:** Cambia el desplazamiento (offset) de lectura/escritura en un archivo.
- **Sintaxis:**

```
off_t lseek(int fd, off_t offset, int whence);
```

- **Parámetros:**
 - fd: Descriptor de archivo.
 - offset: Desplazamiento desde el punto indicado por whence.
 - whence: Punto de referencia (inicio del archivo, posición actual, final).

7. access

- **Propósito:** Verifica los permisos de acceso a un archivo.
- **Sintaxis:**

```
int access(const char *pathname, int mode);
```

- **Parámetros:**
 - pathname: Ruta del archivo a verificar.
 - mode: Permisos a verificar (lectura, escritura, ejecución).

8. stat

- **Propósito:** Recupera información sobre un archivo (metadatos).
- **Sintaxis:**

```
int stat(const char *pathname, struct stat *statbuf);
```

- **Parámetros:**
 - pathname: Ruta del archivo.
 - statbuf: Estructura que almacena los detalles del archivo.

9. chmod

- **Propósito:** Cambia los permisos de un archivo.
- **Sintaxis:**

```
int chmod(const char *pathname, mode_t mode);
```

- **Parámetros:**
 - pathname: Ruta del archivo.
 - mode: Nuevo conjunto de permisos.

10. chown

- **Propósito:** Cambia el propietario y grupo de un archivo.
- **Sintaxis:**

```
int chown(const char *pathname, uid_t owner, gid_t group);
```

- **Parámetros:**
 - pathname: Ruta del archivo.
 - owner: Nuevo ID de usuario propietario.
 - group: Nuevo ID de grupo.

11. fcntl

- **Propósito:** Realiza varias operaciones sobre un descriptor de archivo.
- **Sintaxis:**

```
int fcntl(int fd, int cmd, ... /* arg */);
```

- **Parámetros:**
 - fd: Descriptor de archivo.
 - cmd: Comando a ejecutar (por ejemplo, obtener o establecer flags).
 - arg: Argumento opcional según el comando.

12. chdir

- **Propósito:** Cambia el directorio de trabajo actual.
- **Sintaxis:**

```
int chdir(const char *path);
```

- **Parámetros:**
 - path: Ruta del nuevo directorio de trabajo.

13. mkdir

- **Propósito:** Crea un nuevo directorio.
- **Sintaxis:**

```
int mkdir(const char *pathname, mode_t mode);
```

- **Parámetros:**
 - pathname: Ruta del directorio a crear.
 - mode: Permisos para el nuevo directorio.

14. opendir

- **Propósito:** Abre un directorio para su lectura.
- **Sintaxis:**

```
DIR *opendir(const char *name);
```

- **Parámetros:**
 - name: Ruta del directorio a abrir.

15. readdir

- **Propósito:** Lee las entradas de un directorio abierto.
- **Sintaxis:**

```
struct dirent *readdir(DIR *dirp);
```

- **Parámetros:**
 - dirp: Puntero a un objeto DIR devuelto por opendir.

7. A través del sitio MSDN de Microsoft, investigue y reporte para qué sirven las siguientes llamadas al sistema de Windows: OpenFile, CloseFile, ReadFile, WriteFile, CreateFile, SetFilePointer, CreateDirectory, SetCurrentDirectory, GetFileAttributes, SetFileAttributes, FindFirstFile, y FindNextFile. Investigue y reporte si existe una llamada idéntica a chmod en Windows, en caso de no existir una llamada idéntica, investigar que llamada equivalente existe en Windows.

1. OpenFile:

Abre un archivo. Es una función obsoleta; en su lugar, se recomienda usar CreateFile. Permite abrir un archivo para lectura, escritura o ambos.

Sintaxis: HFILE OpenFile(LPCSTR lpFileName, LPOFSTRUCT lpReOpenBuff, UINT uStyle);

Parámetros: lpFileName (nombre del archivo), lpReOpenBuff (estructura de re-apertura), uStyle (modo de apertura).

2. CloseFile:

No es una función estándar en Windows. Se utiliza CloseHandle para cerrar archivos abiertos con CreateFile.

Sintaxis: BOOL CloseHandle(HANDLE hObject);

Parámetro: hObject (manejador del archivo).

3. ReadFile:

Lee datos de un archivo abierto.

Sintaxis: BOOL ReadFile(HANDLE hFile, LPVOID lpBuffer, DWORD nNumberOfBytesToRead, LPDWORD lpNumberOfBytesRead, LPOVERLAPPED lpOverlapped);

Parámetros: hFile (manejador del archivo), lpBuffer (buffer para almacenar los datos), nNumberOfBytesToRead (número de bytes a leer), lpNumberOfBytesRead (número real de bytes leídos).

4. WriteFile:

Escribe datos en un archivo.

Sintaxis: BOOL WriteFile(HANDLE hFile, LPCVOID lpBuffer, DWORD nNumberOfBytesToWrite, LPDWORD lpNumberOfBytesWritten, LPOVERLAPPED lpOverlapped);

Parámetros: hFile (manejador del archivo), lpBuffer (buffer con datos), nNumberOfBytesToWrite (número de bytes a escribir).

5. CreateFile:

Abre o crea un archivo o dispositivo.

Sintaxis: HANDLE CreateFile(LPCSTR lpFileName, DWORD dwDesiredAccess, DWORD dwShareMode, LPSECURITY_ATTRIBUTES lpSecurityAttributes, DWORD dwCreationDisposition, DWORD dwFlagsAndAttributes, HANDLE hTemplateFile);

Parámetros: lpFileName (nombre del archivo), dwDesiredAccess (acceso de lectura/escritura), dwCreationDisposition (cómo abrir o crear el archivo), etc.

6. SetFilePointer:

Cambia la posición de lectura/escritura en un archivo.

Sintaxis: DWORD SetFilePointer(HANDLE hFile, LONG lDistanceToMove, PLONG lpDistanceToMoveHigh, DWORD dwMoveMethod);

Parámetros: hFile (manejador del archivo), lDistanceToMove (distancia a mover), dwMoveMethod (cómo mover el puntero).

7. CreateDirectory:

Crea un nuevo directorio.

Sintaxis: BOOL CreateDirectory(LPCSTR lpPathName, LPSECURITY_ATTRIBUTES lpSecurityAttributes);

Parámetros: lpPathName (nombre del directorio), lpSecurityAttributes (atributos de seguridad).

8. SetCurrentDirectory:

Establece el directorio actual de trabajo.

Sintaxis: BOOL SetCurrentDirectory(LPCSTR lpPathName);

Parámetro: lpPathName (nuevo directorio de trabajo).

9. GetFileAttributes:

Recupera atributos de un archivo.

Sintaxis: DWORD GetFileAttributes(LPCSTR lpFileName);

Parámetro: lpFileName (nombre del archivo).

10. SetFileAttributes:

Cambia los atributos de un archivo.

Sintaxis: BOOL SetFileAttributes(LPCSTR lpFileName, DWORD dwFileAttributes);

Parámetros: lpFileName (nombre del archivo), dwFileAttributes (nuevos atributos).

11. FindFirstFile:

Busca el primer archivo en un directorio que cumpla con un patrón.

Sintaxis: HANDLE FindFirstFile(LPCSTR lpFileName, LPWIN32_FIND_DATA lpFindFileData);

Parámetros: lpFileName (patrón de búsqueda), lpFindFileData (datos del archivo encontrado).

12. FindNextFile:

Continúa buscando archivos después de FindFirstFile.

Sintaxis: BOOL FindNextFile(HANDLE hFindFile,
LPWIN32_FIND_DATA lpFindFileData);

Parámetros: hFindFile (manejador de la búsqueda), lpFindFileData (datos del archivo encontrado).

8. Utilizando las llamadas al sistema revisadas para Linux que sean necesarias, desarrolle un programa en C que cree una serie aleatoria de archivos (en una ruta especificada a 2 través de la línea de comando, el directorio no debe existir previamente), el contenido de los archivos serán cadenas que estén almacenadas en un arreglo dentro del programa.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <dirent.h>
#include <time.h>
char* contenido()
{
    static const char charset[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    char* contenido = (char*)malloc((50 + 1) * sizeof(char));

    for (int i = 0; i < 50; ++i)
    {
        contenido[i] = charset[rand() % (sizeof(charset) - 1)];
    }

    contenido[50] = '\0';
    return contenido;
}
int main(int argc, char *argv[]){
    if(argc!=2){
        printf("Uso: %s <ruta-del-directorio>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    const char *directory_path = argv[1];
    srand(time(NULL));
    if(mkdir(directory_path,0700)==-1){
        perror("Error al crear directorio");
        exit(EXIT_FAILURE);
    }
}
```

```

    if(chdir(directory_path)==-1){
        perror("Error al cambiar de directorio");
        exit(EXIT_FAILURE);
    }
    for(int i=0;i<5;i++){
        char nombre[20];
        snprintf(nombre, sizeof(nombre), "archivo_%d.txt", i);
        int fd =open(nombre,O_CREAT | O_WRONLY, 0644);
        char* con =contenido();
        if(write(fd,con,strlen(con))==-1){
            perror("Error al escribir en archivo");
            exit(EXIT_FAILURE);
        }
        free(con);
    }
    printf("Archivos aleatorios creados en el directorio: %s\n", directory_path);

    return 0;
}

```

```

said@said-VirtualBox:~/Descargas$ gcc aleatorios.c -o aleatorios
said@said-VirtualBox:~/Descargas$ ./aleatorios '/home/said/Escritorio/Practica2'
Archivos aleatorios creados en el directorio: /home/said/Escritorio/Practica2

```

```

said@said-VirtualBox:~/Escritorio$ cd Practica2
said@said-VirtualBox:~/Escritorio/Practica2$ ls
archivo_0.txt  archivo_1.txt  archivo_2.txt  archivo_3.txt  archivo_4.txt
said@said-VirtualBox:~/Escritorio/Practica2$ 

```



9. Una vez creados los archivos con sus contenidos por el programa del punto 8, y utilizando las llamadas al sistema revisadas para Linux que sean necesarias, desarrolle un programa en C para cambiar los permisos de un archivo seleccionado por el usuario.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <dirent.h>
#include <time.h>
int main(int argc, char *argv[]){
    if(argc!=3){
        printf("Uso: %s <archivo> <permisos>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    char *nombre = argv[1];
    mode_t permiso = strtol(argv[2],NULL,8);
    if(chmod(nombre,permiso)==-1){
        perror("Error cambiar permisos");
        exit(EXIT_FAILURE);
    }

    printf("Permisos cambiados del archivo: %s\n", nombre);

    return 0;
}
```

```
said@said-VirtualBox:~/Escritorio/Practica2$ ls -l archivo_0.txt
-rw-r--r-- 1 said said 50 sep 14 17:38 archivo_0.txt
```

```
said@said-VirtualBox:~/Descargas$ ./permisos '/home/said/Escritorio/Practica2/archivo_0.txt' 543
Permisos cambiados del archivo: /home/said/Escritorio/Practica2/archivo_0.txt
said@said-VirtualBox:~/Descargas$ cd ../Escritorio/Practica2
said@said-VirtualBox:~/Escritorio/Practica2$ ls -l archivo_0.txt
-r-xr--wx 1 said said 50 sep 14 17:38 archivo_0.txt
```

 archivo_0.tx
t

 archivo_1.tx
t

 archivo_2.tx
t

 archivo_3.tx
t

 archivo_4.tx
t

10. Una vez creados los archivos con sus contenidos por el programa del punto 8. y utilizando las llamadas al sistema revisadas para Linux que sean necesarias, desarrolle un programa en C que liste los archivos creados, mostrando su tamaño, fecha y hora de acceso.

```
#include <stdio.h>
#include <stdlib.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>
#include <string.h>
#include <limits.h>
int main(int argc, char *argv[]) {
    struct dirent *d;
    DIR *dir;
    struct stat stati;
    char path[PATH_MAX];
    struct tm *tm;
    char time[100];
    if(argc!=2){
        printf("Uso: %s <ruta-del-directorio>\n", argv[0]);
        exit(EXIT_FAILURE);}
    dir = opendir(argv[1]);
    printf("Archivos en el directorio '%s':\n\n", argv[1]);
    printf("%-25s %-10s %-20s\n", "Nombre del archivo", "Tamaño", "Fecha y hora de último acceso");
    printf("-----\n");
    while ((d = readdir(dir)) != NULL) {
        if (strcmp(d->d_name, ".") == 0 || strcmp(d->d_name, "..") == 0) {
            continue; }
        snprintf(path, sizeof(path), "%s/%s", argv[1], d->d_name);
        if (stat(path, &stati) == -1)
        {
            perror("stat fallido");
            return 1;
        }
    }
}
```

```
    printf("Nombre: %s\n", path);
    printf("Tamaño: %ld bytes\n", stati.st_size);
    printf("Fecha y hora de acceso: %s", ctime(&stati.st_atime));
    printf("\n");
}
closedir(dir);
return 0;
}
```

```
said@said-VirtualBox:~/Descargas$ gcc propiedades.c -o propiedades
said@said-VirtualBox:~/Descargas$ ./propiedades '/home/said/Escritorio/Practica2'
Archivos en el directorio '/home/said/Escritorio/Practica2':
```

Nombre del archivo	Tamaño	Fecha y hora de último acceso
Nombre: /home/said/Escritorio/Practica2/archivo_4.txt	Tamaño: 50 bytes	Fecha y hora de acceso: Sat Sep 14 17:44:29 2024
Nombre: /home/said/Escritorio/Practica2/archivo_3.txt	Tamaño: 50 bytes	Fecha y hora de acceso: Sat Sep 14 17:44:16 2024
Nombre: /home/said/Escritorio/Practica2/archivo_2.txt	Tamaño: 50 bytes	Fecha y hora de acceso: Sat Sep 14 17:38:21 2024
Nombre: /home/said/Escritorio/Practica2/archivo_1.txt	Tamaño: 50 bytes	Fecha y hora de acceso: Sat Sep 14 17:44:07 2024
Nombre: /home/said/Escritorio/Practica2/archivo_0.txt	Tamaño: 50 bytes	Fecha y hora de acceso: Sat Sep 14 20:08:37 2024

11. Una vez creados los archivos con sus contenidos por el programa del punto 8 y utilizando las llamadas al sistema revisadas para Linux que sean necesarias, desarrolle un programa en C para mostrar el contenido de un archivo seleccionado por el usuario.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <dirent.h>
#include <errno.h>
#include <time.h>
#define BUF 1024
void copiar(const char *s, const char *d) {
    int sfd, dfd;
    ssize_t br, bw;
    char bufer[BUF];
    sfd = open(s, O_RDONLY);
    if (sfd == -1) {
        perror("Error al abrir el archivo fuente");
        exit(EXIT_FAILURE);
    }
    dfd = open(d, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dfd == -1) {
        perror("Error al crear el archivo destino");
        close(sfd);
        exit(EXIT_FAILURE);
    }
}
```

```

        while ((br = read(sfd, bufer, BUF)) > 0) {
            bw = write(dfd, bufer, br);
            if (bw == -1) {
                perror("Error al escribir en el archivo destino");
                close(sfd);
                close(dfd);
                exit(EXIT_FAILURE);
            }
        }
        close(sfd);
        close(dfd);
        printf("Archivo %s copiado a %s exitosamente.\n", s, d);
    }
}

int main(int argc, char *argv[]) {
    ssize_t br;
    char bufer[BUF];
    if (argc != 4) {
        printf("Uso: %s <ruta-del-directorio> <nombre-del-archivo> <directorio-destino>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    const char *dir = argv[1];
    const char *file = argv[2];
    const char *destdir = argv[3];
    if (mkdir(destdir, 0755) == -1) {
        if (errno != EEXIST) {
            perror("Error al crear el directorio destino");
            exit(EXIT_FAILURE);
        }
    }
}

```

```

        if (chdir(dir) == -1) {
            perror("Error al cambiar de directorio");
            exit(EXIT_FAILURE);
        }
        int fd = open(file, O_RDONLY);
        if (fd == -1) {
            perror("Error al abrir el archivo");
            exit(EXIT_FAILURE);
        }
        printf("Contenido del archivo %s:\n", file);
        while ((br = read(fd, bufer, BUF)) > 0) {
            write(STDOUT_FILENO, bufer, br);
        }
        if (br == -1) {
            perror("Error al leer el archivo");
        }
        close(fd);
        char dest_path[256];
        snprintf(dest_path, sizeof(dest_path), "%s/%s", destdir, file);
        copiar(file, dest_path);
        return 0;
    }
}

```

```
said@said-VirtualBox:~/Descargas$ gcc contenido.c -o contenido
said@said-VirtualBox:~/Descargas$ ./contenido '/home/said/Escritorio/Practica2'
archivo_0.txt '/home/said/Escritorio/Practica2Copia'
Contenido del archivo archivo_0.txt:
obId0zCXAvE7jG9GIcL39PZAcHZV5tju3kiqK5R5zMe5JIBKkArchivo archivo_0.txt copiado
a /home/said/Escritorio/Practica2Copia/archivo_0.txt exitosamente.
said@said-VirtualBox:~/Descargas$ ./contenido '/home/said/Escritorio/Practica2'
archivo_1.txt '/home/said/Escritorio/Practica2Copia'
Contenido del archivo archivo_1.txt:
uJZk7ZRWMf3TyW4GaCAPyXrAT17kBfD82N5SsQq2TjLf3DM46Archivo archivo_1.txt copiado
a /home/said/Escritorio/Practica2Copia/archivo_1.txt exitosamente.
```



compartida



Practica2



Practica2Co
pia

```
said@said-VirtualBox:~/Descargas$ cd ../Escritorio/Practica2Copia
said@said-VirtualBox:~/Escritorio/Practica2Copia$ ls
archivo_0.txt  archivo_1.txt
```



Ca... nal / Escritorio / Practica2Copia



archivo_0.tx
t



archivo_1.tx
t

Abrir ▾



archivo_0.txt
~/Escritorio/Practica2Copia



pbId0zCXAvE7jG9GIcL39PZAcHZV5tju3kiqK5R5zMe5JIBKk

12. Desarrolle las versiones para Windows de los programas del punto 8 al 11, utilizando las llamadas al sistema para Windows que sean necesarias revisadas en el punto 7.

ALEATORIOSW.C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <time.h>
char* contenido()
{
    static const char charset[] = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    char* contenido = (char*)malloc((50 + 1) * sizeof(char));

    for (int i = 0; i < 50; ++i)
    {
        contenido[i] = charset[rand() % (sizeof(charset) - 1)];
    }

    contenido[50] = '\0';
    return contenido;
}
int main(int argc, char *argv[]){
    if(argc!=2){
        printf("Uso: %s <ruta-del-directorio>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    const char *directory_path = argv[1];
    srand(time(NULL));
    if(CreateDirectory(directory_path,NULL)==0){
        perror("Error al crear directorio");
        exit(EXIT_FAILURE);
    }
    if(SetCurrentDirectory(directory_path)==0){
        perror("Error al cambiar de directorio");
        exit(EXIT_FAILURE);
    }

    for(int i=0;i<5;i++){
        char nombre[20];
        sprintf(nombre, sizeof(nombre), "archivo_%d.txt", i);
        HANDLE fd = CreateFile(nombre, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
        char* con =contenido();
        DWORD bytes;
        if(WriteFile(fd,con,strlen(con),&bytes,NULL)==-1){
            perror("Error al escribir en archivo");
            exit(EXIT_FAILURE);
        }
        if (CloseHandle(fd) == 0)
        {
            perror("Fallo al cerrar el archivo");
            exit(EXIT_FAILURE);
        }
        free(con);
    }
    printf("Archivos aleatorios creados en el directorio: %s\n", directory_path);

    return 0;
}
```

```

C:\Users\vbcs1>gcc aleatoriosw.c -o aleatorios

C:\Users\vbcs1>aleatorios \Users\vbcs1\Practica2
Archivos aleatorios creados en el directorio: \Users\vbcs1\Practica2

C:\Users\vbcs1>cd Practica2

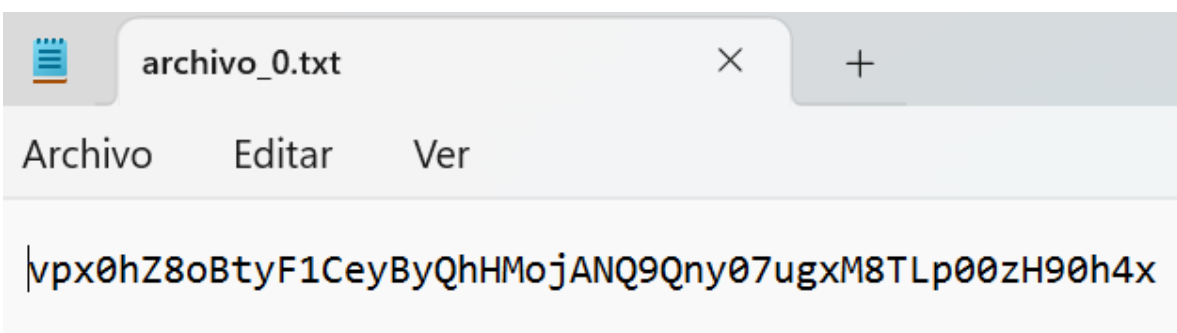
C:\Users\vbcs1\Practica2>dir
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150

Directorio de C:\Users\vbcs1\Practica2

15/09/2024  14:09    <DIR>          .
15/09/2024  14:09    <DIR>          ..
15/09/2024  14:09                50 archivo_0.txt
15/09/2024  14:09                50 archivo_1.txt
15/09/2024  14:09                50 archivo_2.txt
15/09/2024  14:09                50 archivo_3.txt
15/09/2024  14:09                50 archivo_4.txt
               5 archivos                250 bytes
               2 dirs 382.062.497.792 bytes libres

```

	Practica2	15/09/2024 14:09	Carpeta de archivos
	archivo_0	15/09/2024 14:09	Documento de texto
	archivo_1	15/09/2024 14:09	Documento de texto
	archivo_2	15/09/2024 14:09	Documento de texto
	archivo_3	15/09/2024 14:09	Documento de texto
	archivo_4	15/09/2024 14:09	Documento de texto



PERMISOSW.C

```
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
int main(int argc, char *argv[]){
    if(argc!=3){
        printf("Uso: %s <archivo> <permisos>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    char *nombre = argv[1];
    DWORD permisos = atoi(argv[2]);
    if(!SetFileAttributes(nombre, permisos)){
        perror("Error cambiar permisos");
        exit(EXIT_FAILURE);
    }

    printf("Permisos cambiados del archivo: %s\n", nombre);

    return 0;
}
```

```
C:\Users\vbcs1>permisos \Users\vbcs1\Practica2\archivo_0.txt 7
Permisos cambiados del archivo: \Users\vbcs1\Practica2\archivo_0.txt

C:\Users\vbcs1> cd Practica2

C:\Users\vbcs1\Practica2>icacls archivo_0.txt
archivo_0.txt NT AUTHORITY\SYSTEM:(I)(F)
                BUILTIN\Administradores:(I)(F)
                LAPTOP-65JBVCKH\vbcs1:(I)(F)

Se procesaron correctamente 1 archivos; error al procesar 0 archivos
```

PROPIEDADESW.C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <tchar.h>

void imprimirFecha(FILETIME ft) {
    SYSTEMTIME st;
    FileTimeToSystemTime(&ft, &st);
    printf("%02d/%02d/%d %02d:%02d:%02d\n", st.wDay, st.wMonth, st.wYear, st.wHour, st.wMinute, st.wSecond);
}

int main(int argc, char *argv[]) {
    WIN32_FIND_DATA fd;
    HANDLE h;
    char s[MAX_PATH];
    if (argc != 2) {
        printf("Uso: %s <ruta-del-directorio>\n", argv[0]);
        return EXIT_FAILURE;
    }
    snprintf(s, sizeof(s), "%s\\*", argv[1]);
    h = FindFirstFile(s, &fd);
    if (h == INVALID_HANDLE_VALUE) {
        printf("No se pudo abrir el directorio %s\n", argv[1]);
        return EXIT_FAILURE;
    }
    printf("Archivos en el directorio '%s':\n\n", argv[1]);
    printf("%-25s %-10s %-20s\n", "Nombre del archivo", "Tam", "Fecha y hora de ultimo acceso");
    printf("-----\n");
```

```
        do {
            if (strcmp(fd.cFileName, ".") != 0 && strcmp(fd.cFileName, "..") != 0) {
                printf("%-25s ", fd.cFileName);
                LARGE_INTEGER fs;
                fs.LowPart = fd.nFileSizeLow;
                fs.HighPart = fd.nFileSizeHigh;
                printf("%-10lld ", fs.QuadPart);
                imprimirFecha(fd.ftLastAccessTime);
            }
        } while (FindNextFile(h, &fd) != 0);
    FindClose(h);
    return 0;
}
```

```
C:\Users\vbcs1>gcc propiedadesw.c -o propiedades
```

```
C:\Users\vbcs1>propiedades \Users\vbcs1\Practica2
Archivos en el directorio '\Users\vbcs1\Practica2':
```

Nombre del archivo	Tam	Fecha y hora de ultimo acceso

archivo_0.txt	50	15/09/2024 20:13:31
archivo_1.txt	50	15/09/2024 20:09:27
archivo_2.txt	50	15/09/2024 20:09:27
archivo_3.txt	50	15/09/2024 20:09:27
archivo_4.txt	50	15/09/2024 20:09:27

CONTENIDOW.C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#define BUF 1024
void copiar(const char *s, const char *d) {
    HANDLE hs, hd;
    DWORD br, bw;
    char bufer[BUF];
    hs = CreateFile(s, GENERIC_READ, 0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hs == INVALID_HANDLE_VALUE) {
        printf("Error al abrir el archivo fuente: %s\n", s);
        return;
    }
    hd = CreateFile(d, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hd == INVALID_HANDLE_VALUE) {
        printf("Error al crear el archivo destino: %s\n", d);
        CloseHandle(hs);
        return;
    }
    while (ReadFile(hs, bufer, BUF, &br, NULL) && br > 0) {
        if (!WriteFile(hd, bufer, br, &bw, NULL) || bw != br) {
            printf("Error al escribir en el archivo destino.\n");
            CloseHandle(hs);
            CloseHandle(hd);
            return;
        }
    }
    CloseHandle(hs);
    CloseHandle(hd);
    printf("Archivo %s copiado a %s exitosamente.\n", s, d);
}
```

```
int main(int argc, char *argv[]) {
    DWORD br;
    char bufer[BUF];
    if (argc != 4) {
        printf("Uso: %s <ruta-del-directorio> <nombre-del-archivo> <directorio-destino>\n", argv[0]);
        return EXIT_FAILURE;
    }
    const char *dir = argv[1];
    const char *file = argv[2];
    const char *destdir = argv[3];
    if (!CreateDirectory(destdir, NULL)) {
        if (GetLastError() != ERROR_ALREADY_EXISTS) {
            printf("Error al crear el directorio destino.\n");
            return EXIT_FAILURE;
        }
    }
    if (!SetCurrentDirectory(dir)) {
        printf("Error al cambiar de directorio a: %s\n", dir);
        return EXIT_FAILURE;
    }
    HANDLE hf = CreateFile(file, GENERIC_READ, 0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hf == INVALID_HANDLE_VALUE) {
        printf("Error al abrir el archivo: %s\n", file);
        return EXIT_FAILURE;
    }
    printf("Contenido del archivo %s:\n", file);
    while (ReadFile(hf, bufer, BUF, &br, NULL) && br > 0) {
        fwrite(bufer, 1, br, stdout);
    }
    if (GetLastError() != ERROR_HANDLE_EOF) {
        printf("Error al leer el archivo.\n");
    }
}
```

```
CloseHandle(hf);
char dest_path[MAX_PATH];
snprintf(dest_path, sizeof(dest_path), "%s\\%s", destdir, file);
copiar(file, dest_path);
return 0;
}
```

```
C:\Users\vbcs1>contenido \Users\vbcs1\Practica2 archivo_0.txt \Users\vbcs1\Practica2Copia
Contenido del archivo archivo_0.txt:
vpx0hZ8oBtyF1CeyByQhHMoJANQ9Qny07ugxM8TLp00zH90h4xError al leer el archivo.
Archivo archivo_0.txt copiado a \Users\vbcs1\Practica2Copia\archivo_0.txt exitosamente.
```

```
C:\Users\vbcs1>contenido \Users\vbcs1\Practica2 archivo_1.txt \Users\vbcs1\Practica2Copia
Contenido del archivo archivo_1.txt:
HtNwVF1FIZwEwp8HrHVogKuWQOk1dQ4vNVs2hNAmWTbGUkoivQError al leer el archivo.
Archivo archivo_1.txt copiado a \Users\vbcs1\Practica2Copia\archivo_1.txt exitosamente.
```

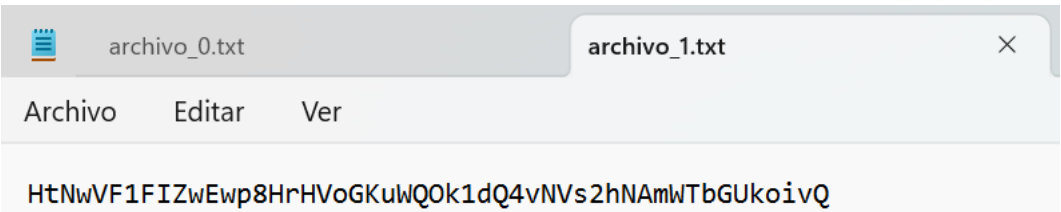
```
C:\Users\vbcs1>contenido \Users\vbcs1\Practica2 archivo_4.txt \Users\vbcs1\Practica2Copia
Contenido del archivo archivo_4.txt:
fq6HUKdaoJzGbYAwLfz0wOF0g6jpPcyuiCU0VfFGZvjmkUMquxError al leer el archivo.
Archivo archivo_4.txt copiado a \Users\vbcs1\Practica2Copia\archivo_4.txt exitosamente.
```

```
C:\Users\vbcs1>cd Practica2Copia

C:\Users\vbcs1\Practica2Copia>dir
El volumen de la unidad C es Windows-SSD
El número de serie del volumen es: C6A9-0150

Directorio de C:\Users\vbcs1\Practica2Copia
15/09/2024  15:01    <DIR>          .
15/09/2024  14:59    <DIR>          ..
15/09/2024  14:59                50 archivo_0.txt
15/09/2024  15:01                50 archivo_1.txt
15/09/2024  15:01                50 archivo_4.txt
               3 archivos                150 bytes
               2 dirs  379.061.342.208 bytes libres
```

	Practica2	15/09/2024 14:09	Carpeta de archivos
	Practica2Copia	15/09/2024 15:01	Carpeta de archivos
	archivo_0	15/09/2024 14:59	Documento de texto
	archivo_1	15/09/2024 15:01	Documento de texto
	archivo_4	15/09/2024 15:01	Documento de texto



Conclusión

Esta práctica nos permitió como equipo introducirnos ya no solo de manera teórica sino también práctica al uso y conocimiento de ¿qué es? y ¿cómo se usa? Un sistema operativo. También nos ayudó a volver a ocupar un lenguaje de programación distinto a otros que más utilizamos a diario como es el lenguaje C, así también reconocer otro tipo de distribuciones de Linux y su gran gama de comandos para poder usar de forma mas eficiente este sistema operativo, sus principales características y diferenciaciones entre lo que es un sistema operativo de código cerrado (Windows) y otro de código abierto como puede ser Linux, en fin, una practica interesante en este inicio de curso y bastante importante para introducirnos en este rubro del Hardware y del Software.